HACKATHON MANAGEMENT SYSTEM – User Requirements

The Hackathon Management System is a web based platform designed to facilitate the entire lifecycle of software competitions. The system is used by multiple types of users who interact with it according to their responsibilities and permissions. A user begins interaction with the system by creating an account .To ensure proper identity verification, all users must provide their first name, surname and age along with using a unique email address and password during registration. After successful authentication, the system identifies the user's role and presents only the functionalities allowed for that role. This ensures that organizers, participants, judges, and mentors operate strictly within their authorization boundaries.

The lifecycle of a hackathon begins with the organizer . They are responsible for creating hackathon events, defining key details such as name, duration, start date, description, and location. Organizers manage the event's lifecycle by controlling its status (e.g., publishing or cancelling the event, etc.) . Once an event is created, the organizer defines one or more challenges , each identified by a unique title within that event , that participants will work on. Challenges are an integral part of the event and cannot exist independently. If an event is cancelled, all challenges related to it are removed automatically by the system. To encourage competition, organizers also define rewards (e.g., prizes or certificates) specific to that event. Similar to challenges, rewards are strictly tied to the event's lifecycle, they cannot exist independently and are removed if the event is cancelled. Rewards will be granted to the highest scoring teams at the end of the event.

Participants join the system to take part in hackathon events. They cannot participate individually; instead, they must either create a new team or join an existing one. A team is identified by a team name, creation date, and number of team members. To manage resources, teams are enforced to have a maximum capacity. Furthermore, within each team, one participant is designated as the team lead to manage submissions. To ensure fair play, the system enforces a strict rule: a participant acts as a member of only one team for any single active event. They cannot compete in multiple teams simultaneously. Teams collaborate on solving challenges and submit their solutions as projects . Submissions include a project link, project name, and a timestamp, which must be within the event's active duration . Since modern projects rely on various technologies, the system also records the tools (e.g., libraries, frameworks) used in each submission. Crucially, the system tracks exactly which team used which tool version , allowing for detailed technical analysis of technology trends across competitions.

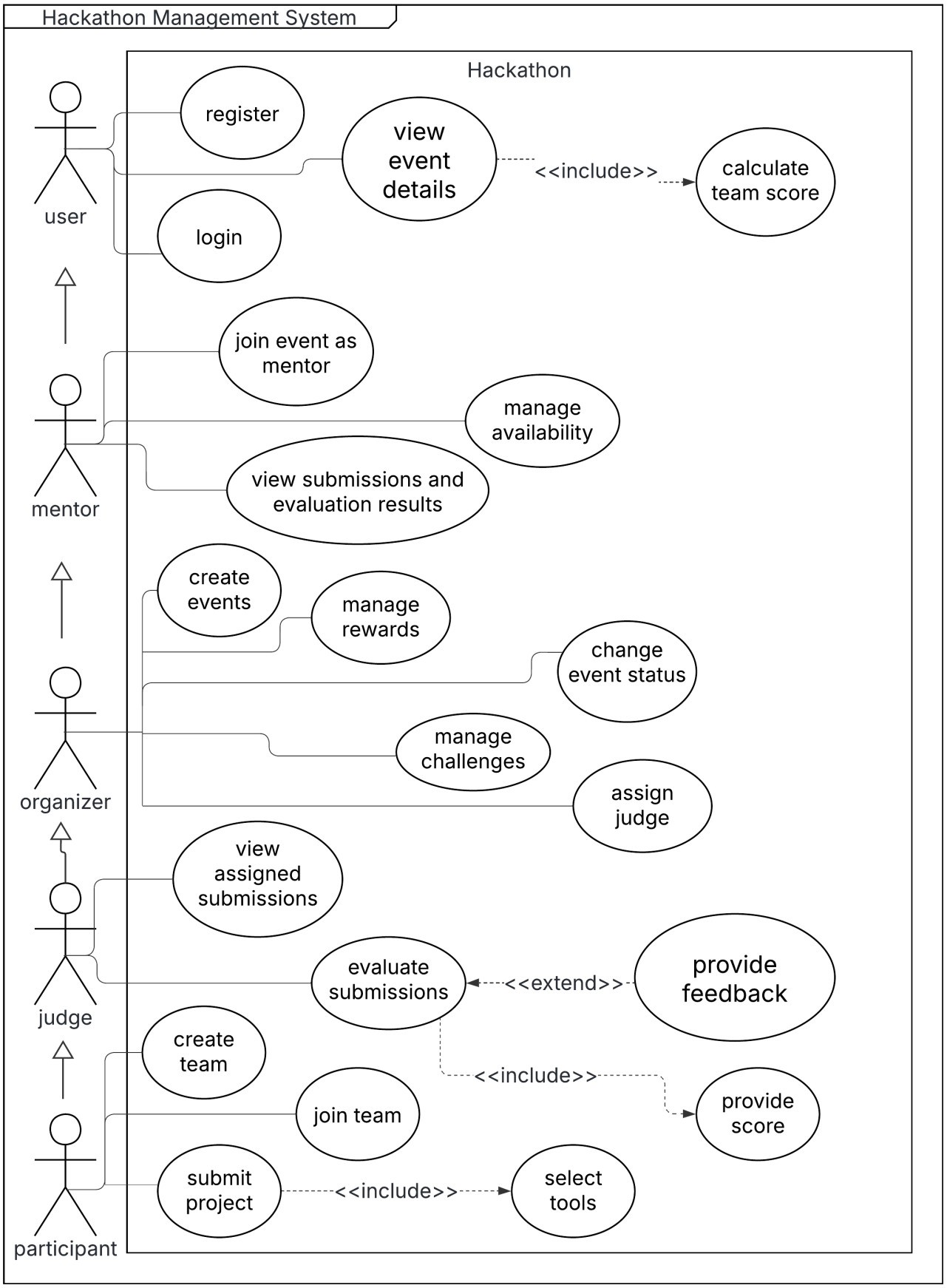
Mentors support participants throughout the event by providing guidance on challenges and reviewing submissions and evaluation results. They sign up to assist specifically for certain events, and the system tracks their available hours for each event they join. Mentors have read only access to submissions and evaluation results and are not allowed to score or modify any project data. Similarly, judges are assigned to evaluate competitions, but the system restricts this process: only users who currently hold the judge role can be selected for an event's jury panel. This ensures that unqualified users are never assigned to evaluate projects. Additionally, to enforce ethical standards, a user is restricted from being both a judge and a participant in the same event.

During the evaluation phase, judges review submissions and create evaluations , providing a numeric score and optionally written feedback. To facilitate the workflow, the system tracks the status of each evaluation assignment(e.g. Pending, Completed, Draft). When the organizer first creates an event , it is saved as a Draft allowing modifications before going public. Once finalized, the organizer publishes the event, moving it to the Registration phase where participants can join. The event then transitions to Active/Ongoing at the start date, allowing submissions. After the duration ends, the event automatically enters the Evaluation phase for judging. Finally once results are finalized, the event is marked as Completed.

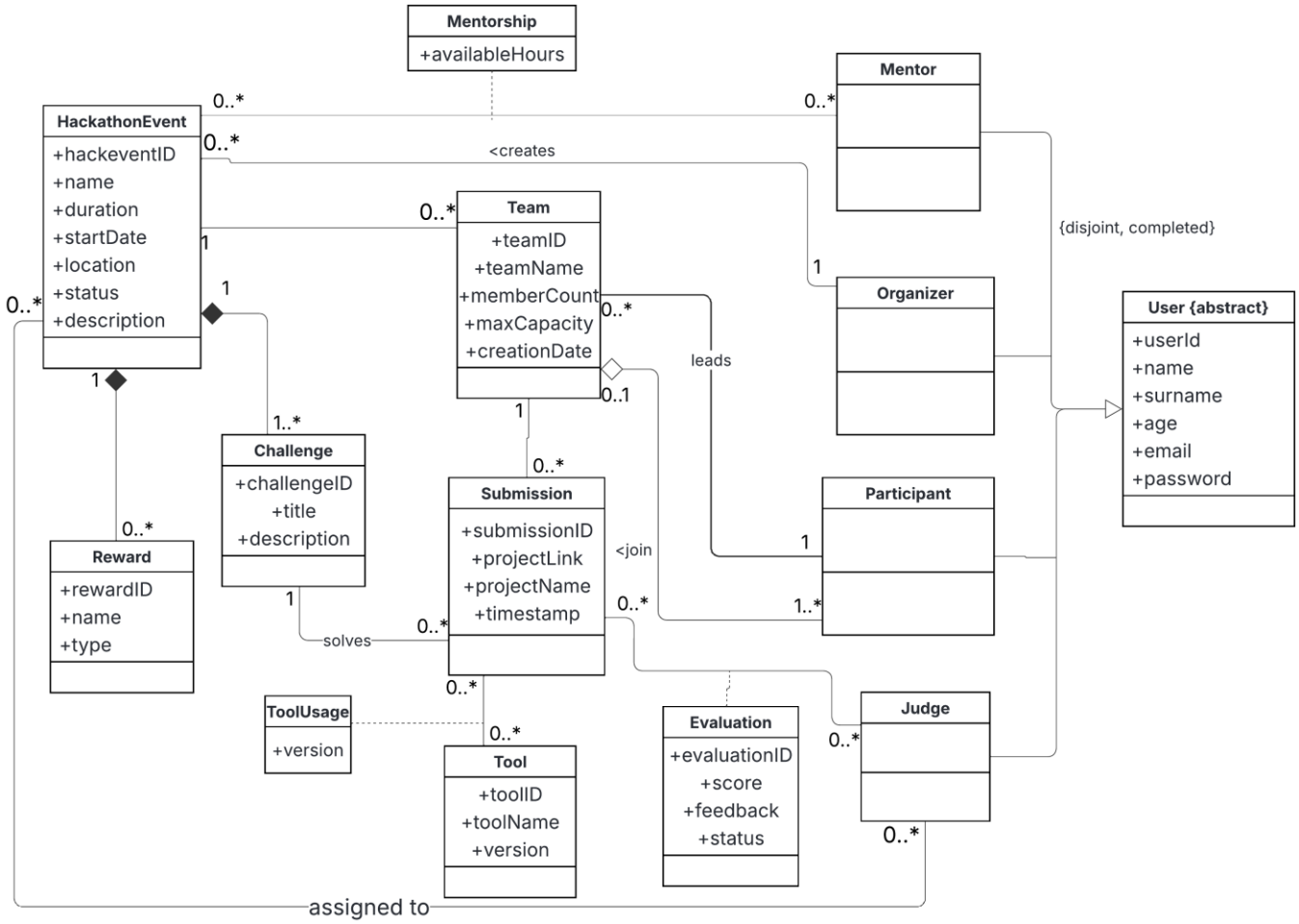
A single submission may be evaluated by multiple judges. The system does not store team scores as fixed values instead, it calculates the team score dynamically by averaging all evaluation scores related to the team's submissions whenever the score is requested.

The system presents information in a hierarchical and consistent manner. Users can select a hackathon event and view its related challenges, submissions, rewards, and teams (including their current standings) without leaving the current context. Whenever event details are viewed, the team scores are calculated dynamically to display up to date standings**.** All actions performed in the system are validated according to business rules to ensure fairness, transparency, and data integrity throughout the hackathon lifecycle.

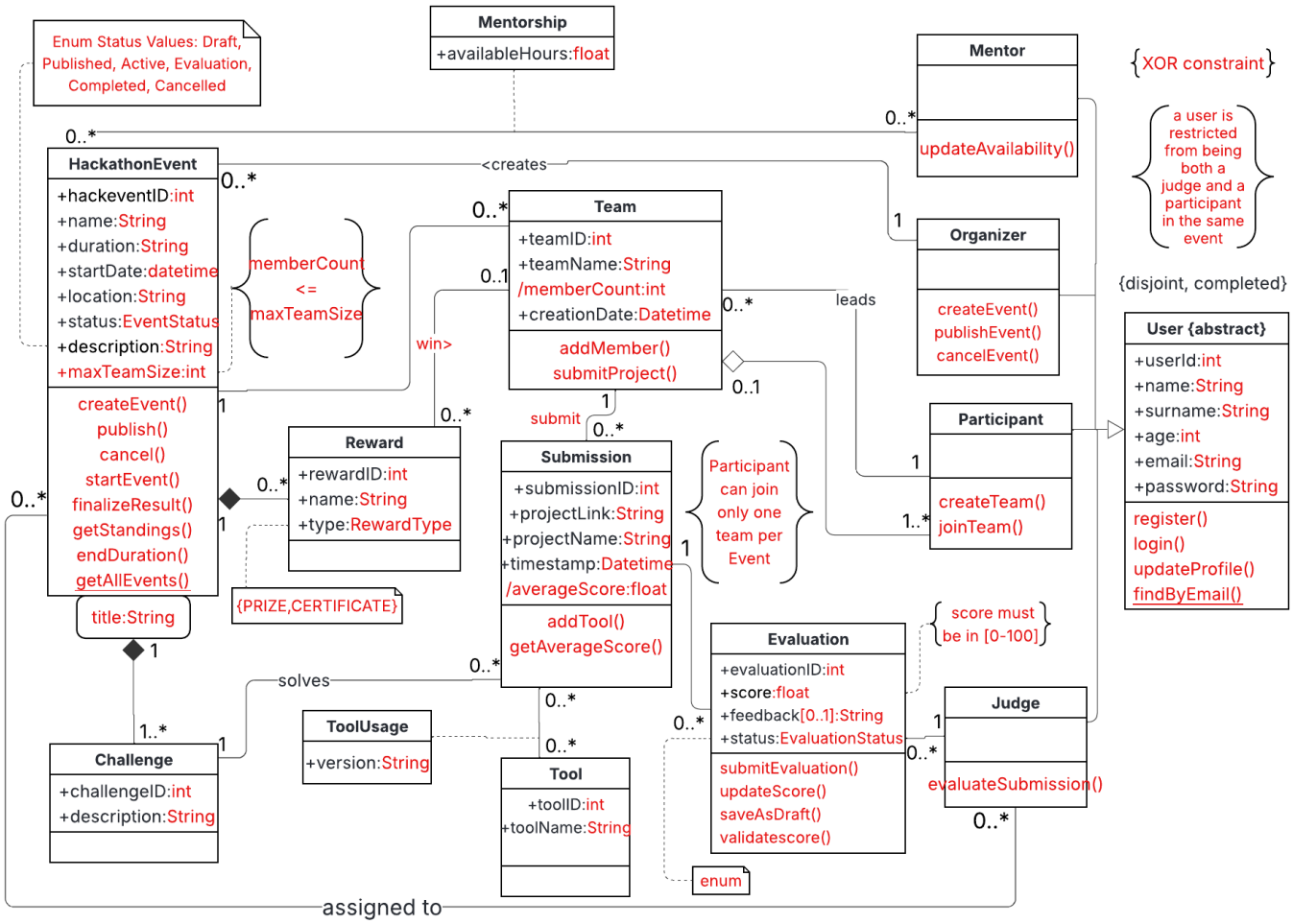
USE CASE DIAGRAM



ANALYTICAL CLASS DIAGRAM



DESIGN CLASS DIAGRAM



USE CASE SCENARIO-EVALUATE SUBMISSION

Use Case Scenario- Evaluate Submission

Actor : Judge

Purpose and context: To assess a submitted project by reviewing its content and creating an evaluation that may include a numeric score and written feedback.

Assumptions:

1.The judge has a stable internet connection to view submission details(External links)

2.The submission link provided by team is active and accessible.

Pre conditions:

1.The event is currently in the active evaluation phase.

2.The specific submission has been assigned to the Judge and its status is Pending.

Trigger:

1. The Judge clicks the "Evaluate" button on a spesific submission listed with "Pending" status on their dashboard.

Basic Flow Of Events:

1.Judge selects the specific submission to evaluate.
2.System displays the submission details(Project name, links, tools used) and the Evaluation form.
3.Judge reviews the submission materials.
4.Judge enters a numeric score (0-100)
5.Judge optionally enters written feedback
6.Judge confirms the evaluation
7.System validates that the mandatory score is present and within allowed range.
8.System saves the evaluation record linked to the Team.
9.System updates the assignment status from Pending to Completed
10.System displays a success message.

Alternative Flow of Events:

7A.Missing Mandatory Score:

1.System detects empty score field
2.System displays error: "Score is mandatory"
3.Flow returns to Step 4

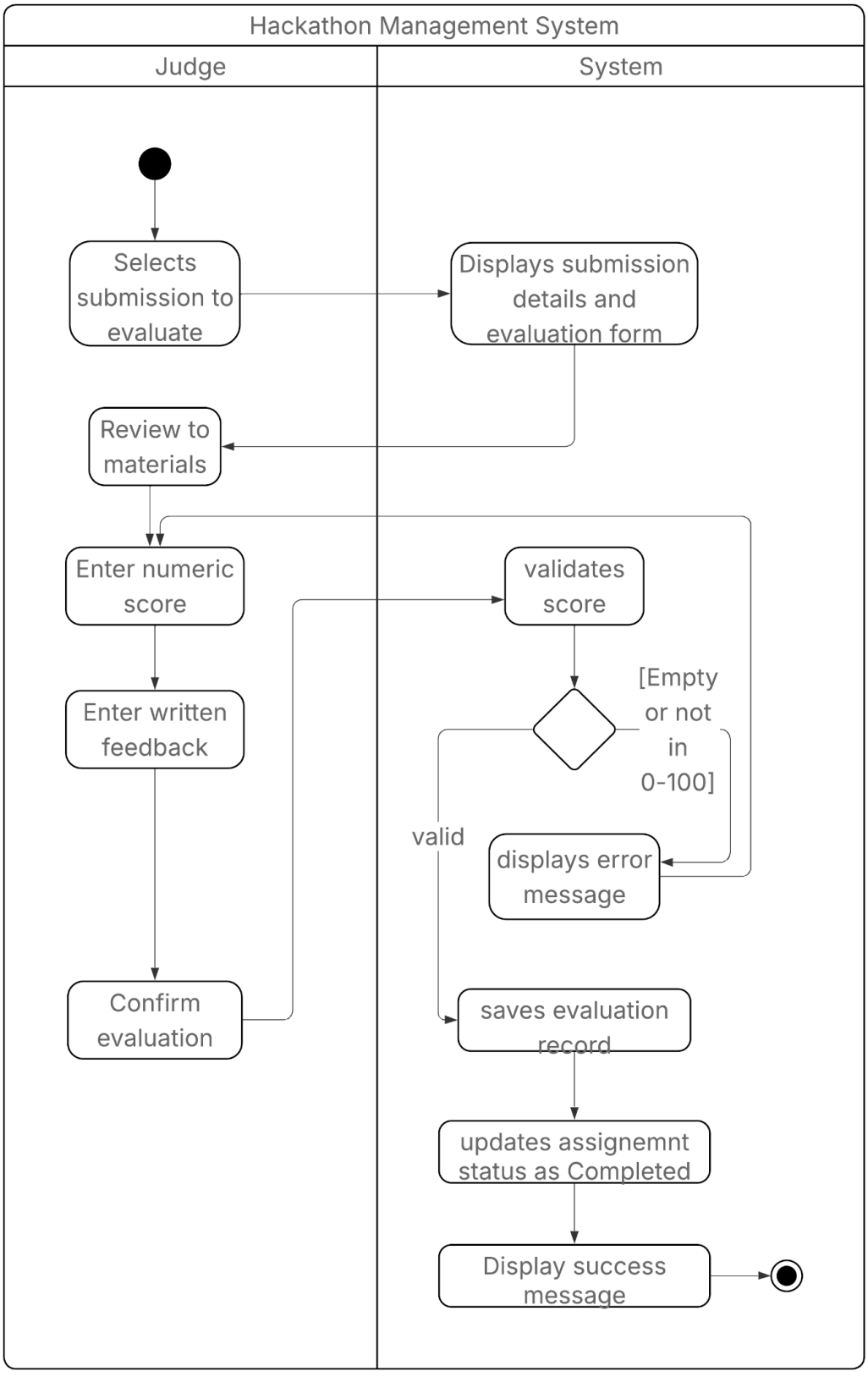
7B. Invalid Score:

1.System detects score is not between 0-100
2.System displays error
3.Flow returns to Step 4

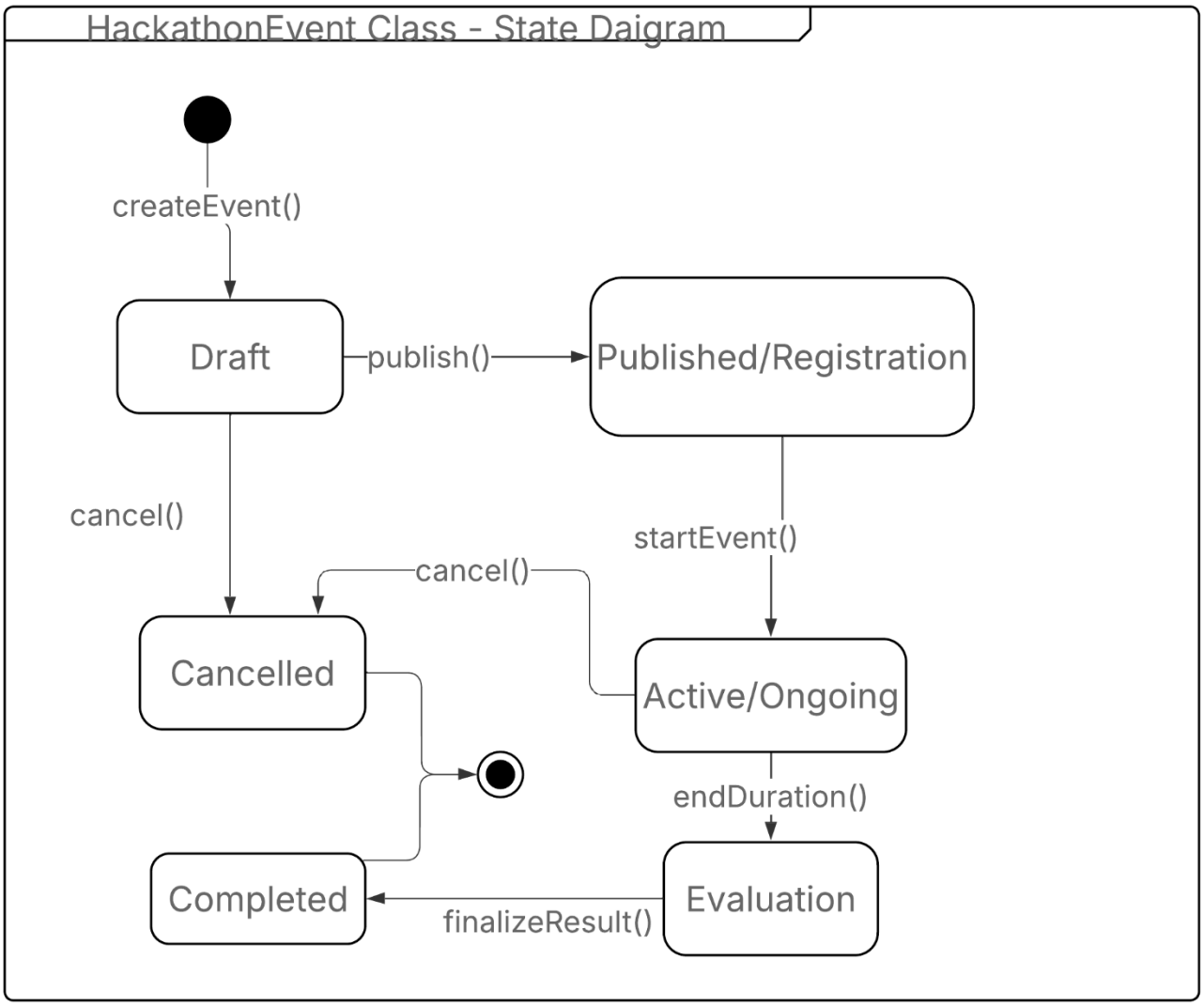
Post conditions:

1. The assingment status is marked as Completed.
2. The evaluation data is persistent
3. The teams dynamic score is updated (ready for calculation)

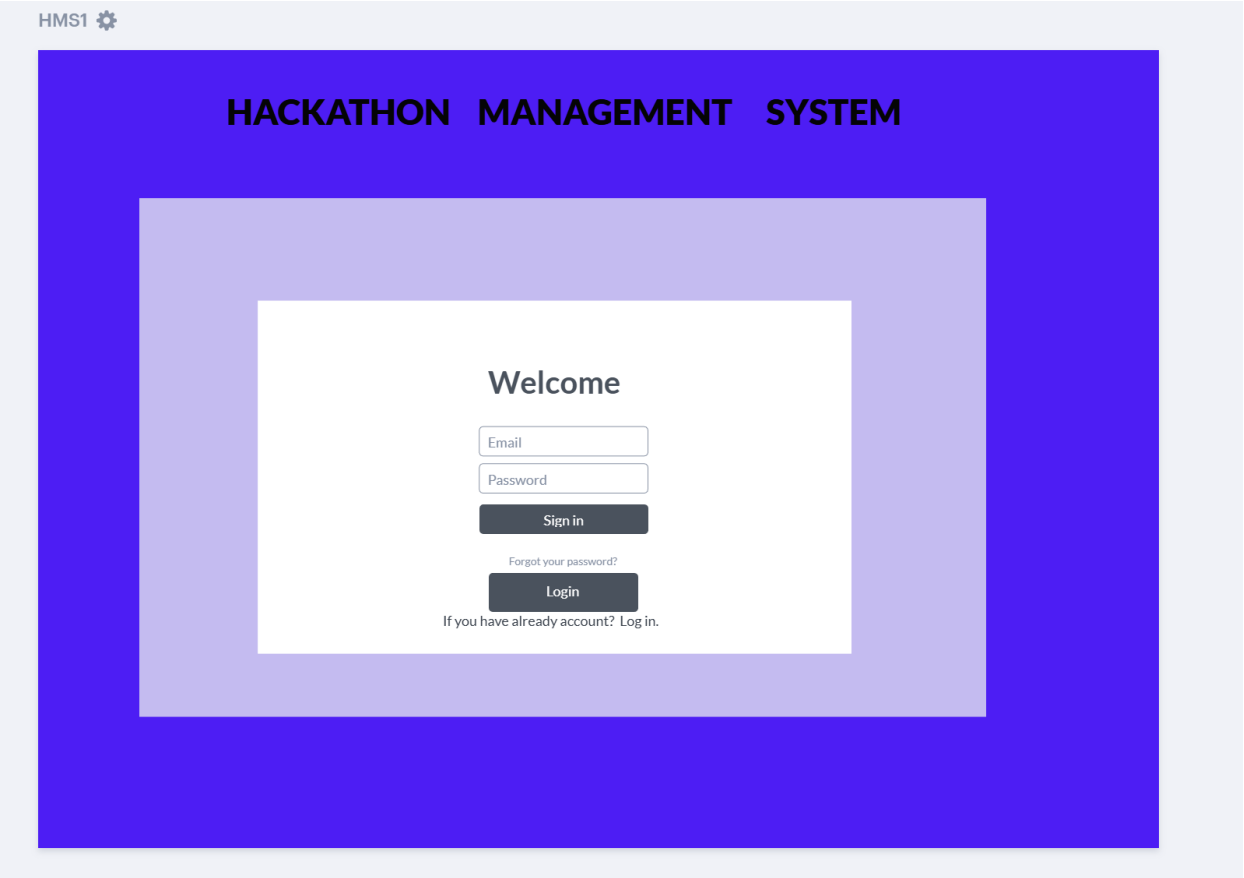
ACTIVITY DIAGRAM- - EVALUATE SUBMISSION

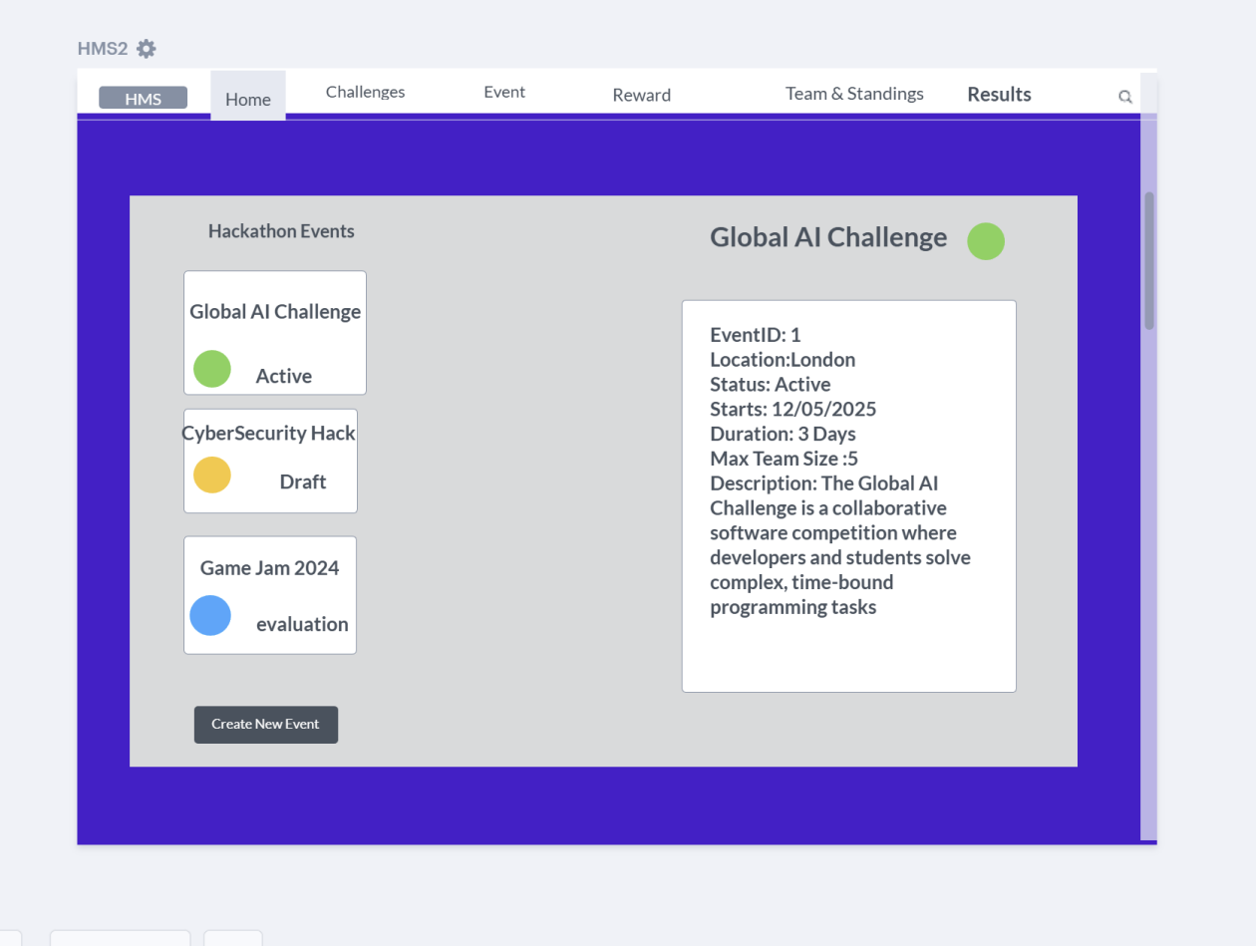


STATE DIAGRAM – HACKATHONEVENT CLASS

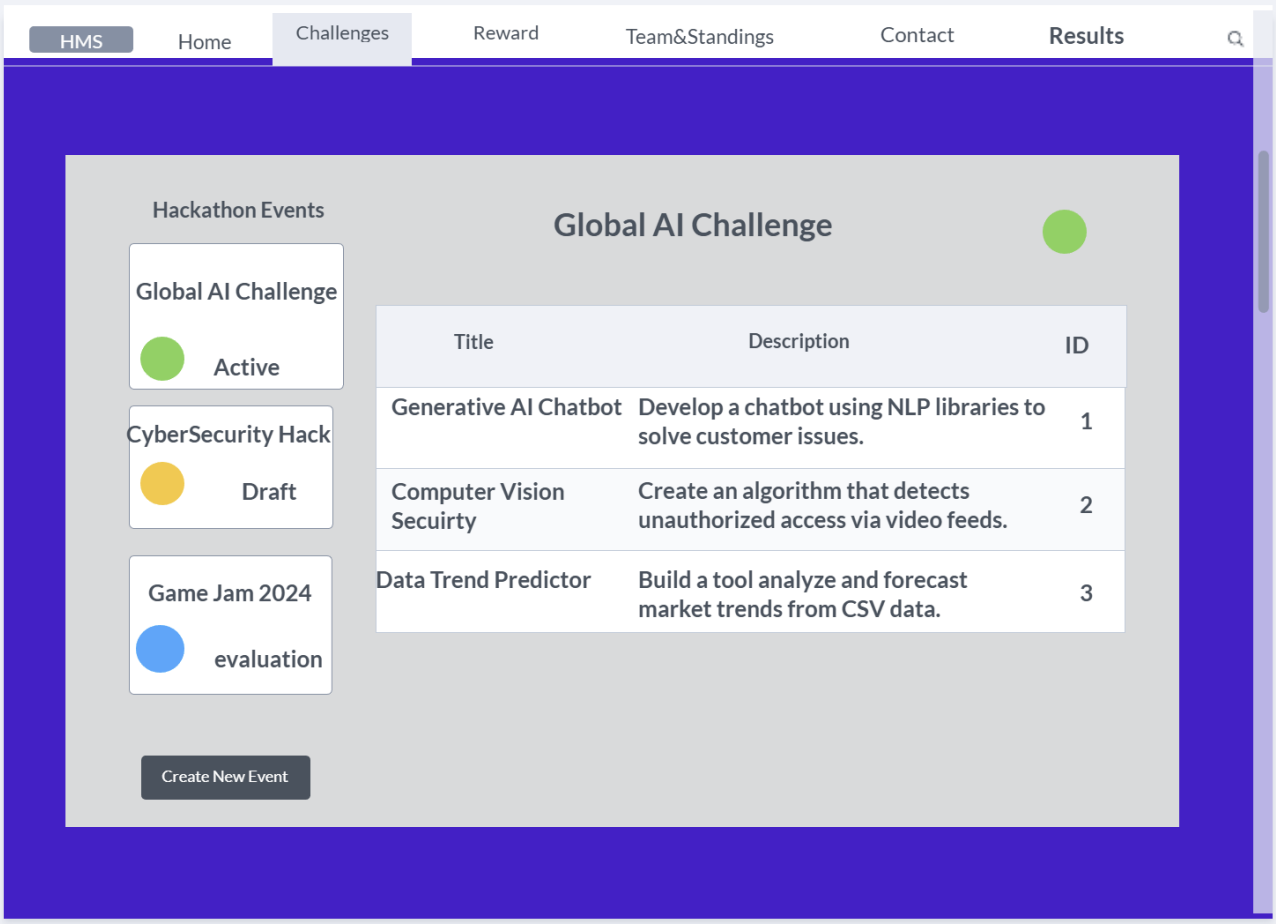


GUI DESIGN

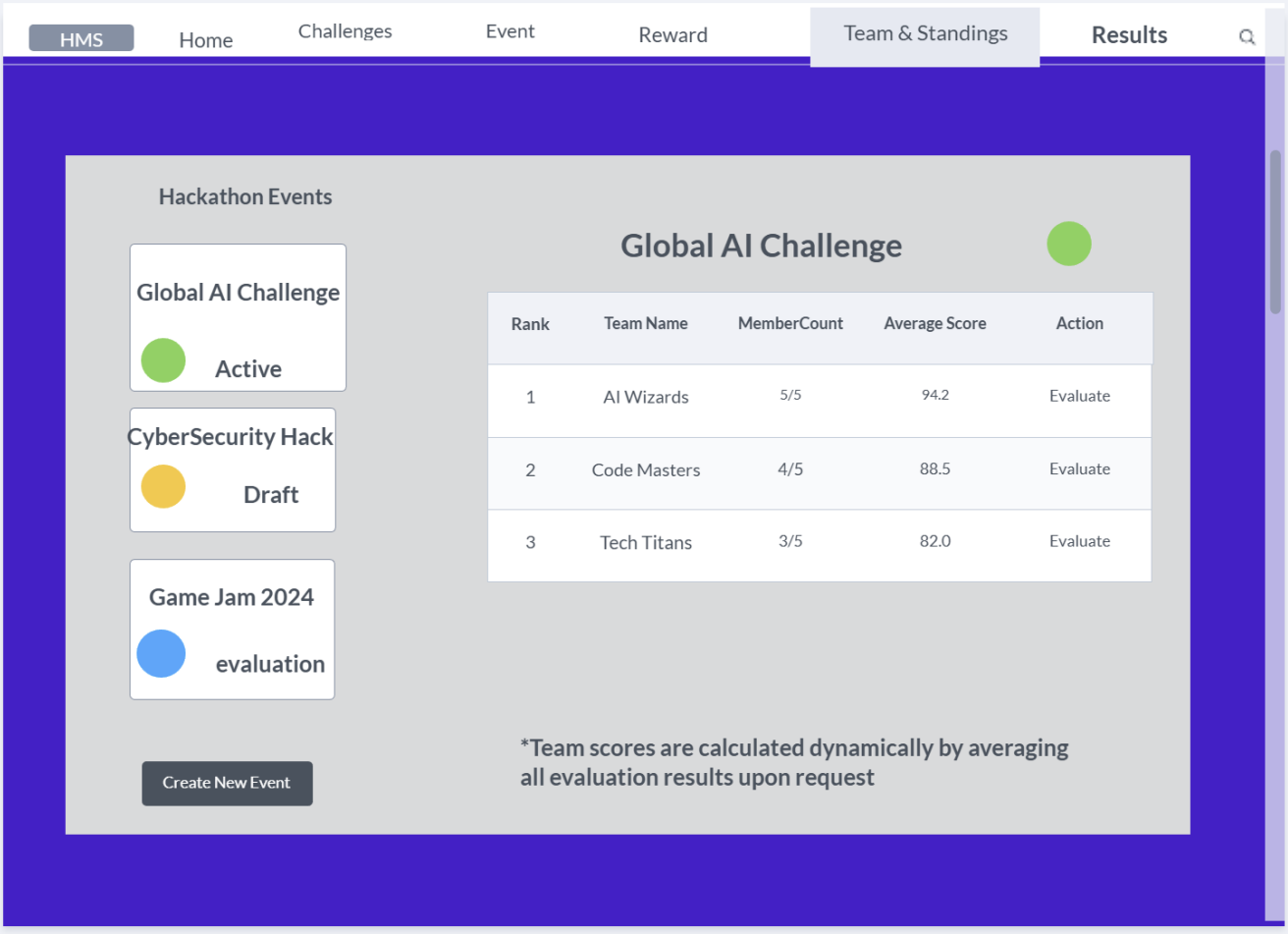

HMS1
HACKATHON MANAGEMENT SYSTEM
Welcome
Email
Password
Sign in
Forgot your password?
Login
If you have already account? Log in.


HMS2
HMS
Home
Challenges
Event
Reward
Team & Standings
Results
Hackathon Events
Global AI Challenge
Active
CyberSecurity Hack
Draft
Game Jam 2024
evaluation
Create New Event
Global AI Challenge
EventID: 1
Location:London
Status: Active
Starts: 12/05/2025
Duration: 3 Days
Max Team Size :5
Description: The Global AI Challenge is a collaborative software competition where developers and students solve complex, time-bound programming tasks

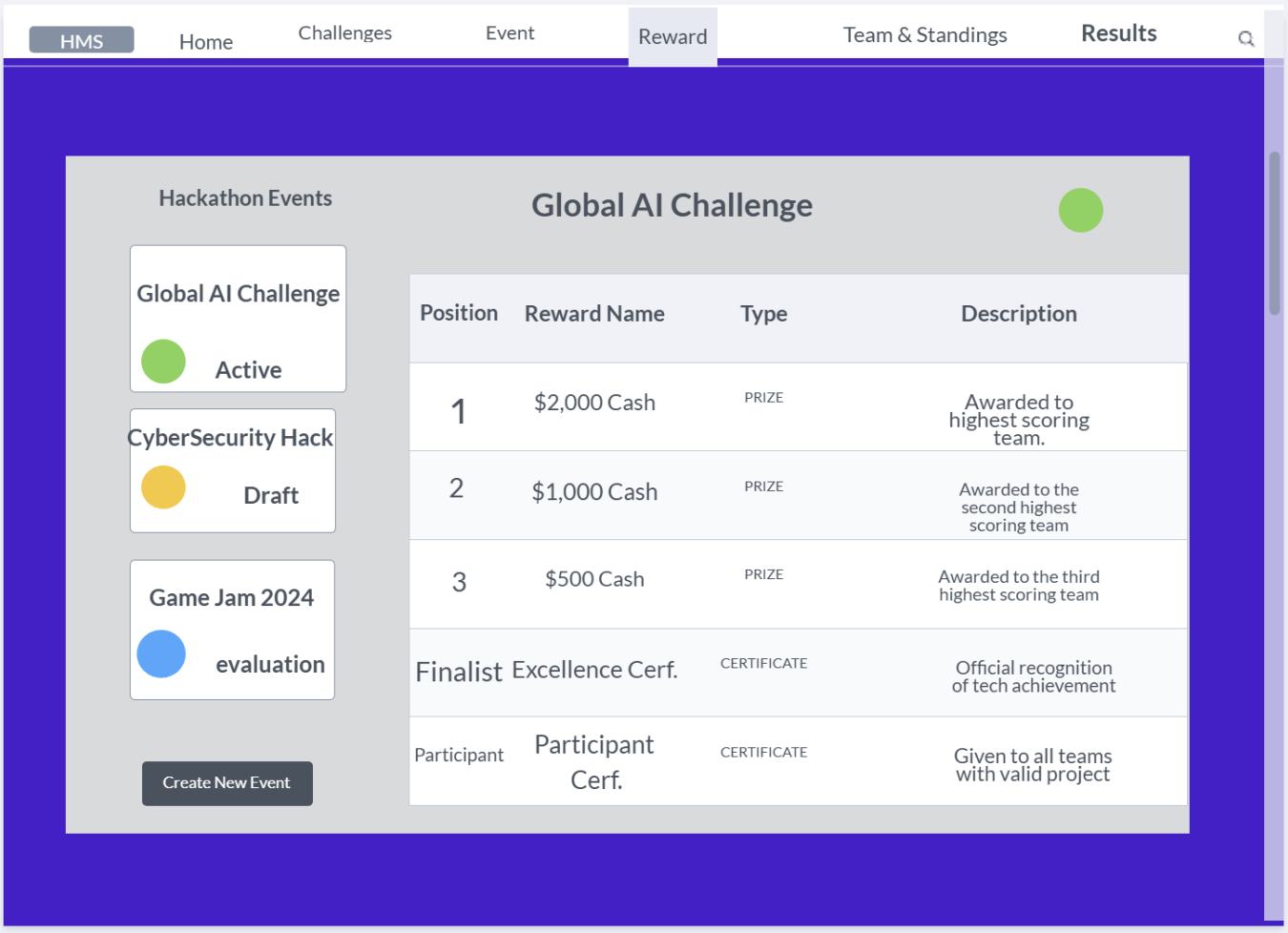
HMS4


HMS
Home
Challenges
Reward
Team&Standings
Contact
Results
Hackathon Events
Global AI Challenge
Active
CyberSecurity Hack
Draft
Game Jam 2024
evaluation
Create New Event
Global AI Challenge
Title
Description
ID
Generative AI Chatbot
Develop a chatbot using NLP libraries to solve customer issues.
1
Computer Vision Secuirty
Create an algorithm that detects unauthorized access via video feeds.
2
Data Trend Predictor
Build a tool analyze and forecast market trends from CSV data.
3

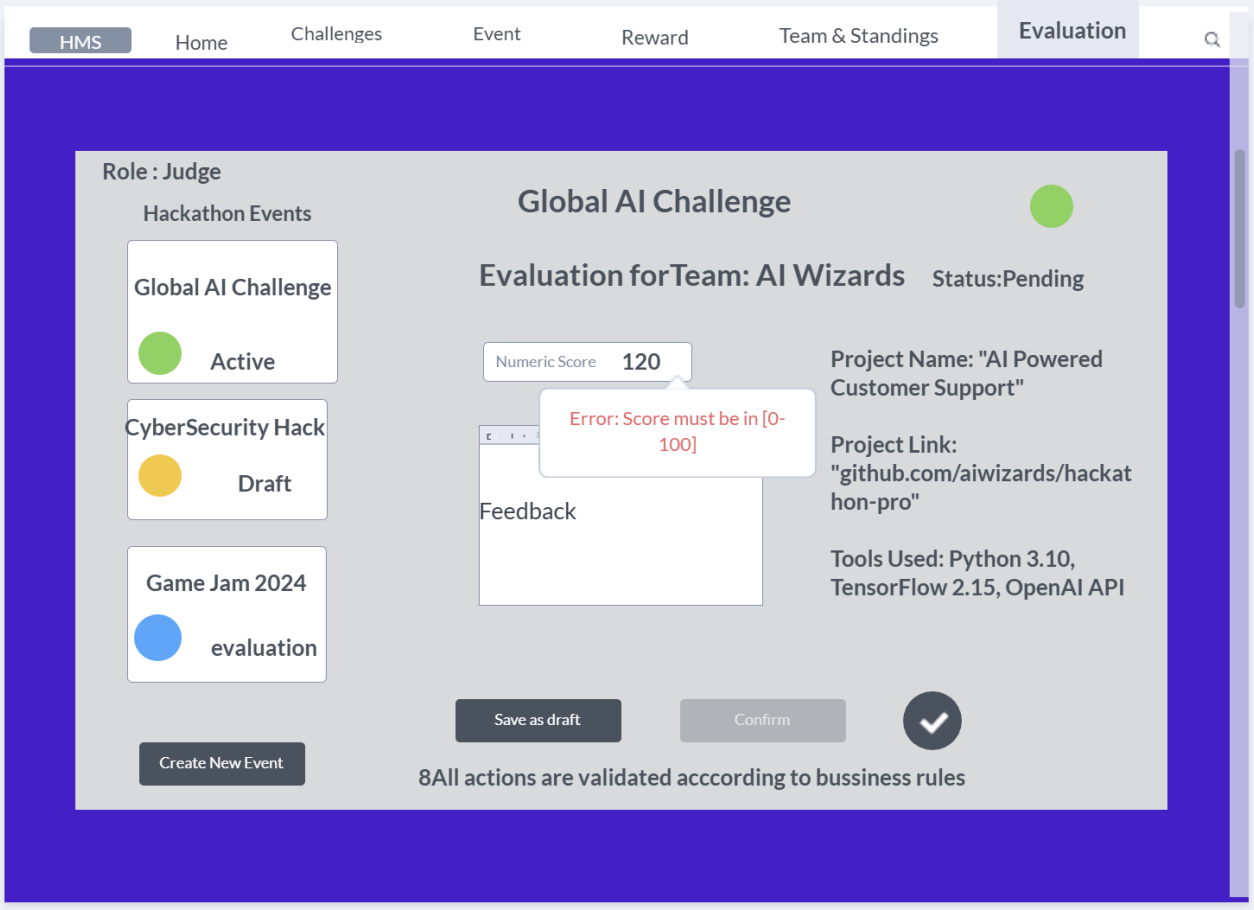
HMS3


HMS
Home
Challenges
Event
Reward
Team & Standings
Results
Hackathon Events
Global AI Challenge
Active
CyberSecurity Hack
Draft
Game Jam 2024
evaluation
Create New Event
Global AI Challenge
Rank
Team Name
MemberCount
Average Score
Action
1
AI Wizards
5/5
94.2
Evaluate
2
Code Masters
4/5
88.5
Evaluate
3
Tech Titans
3/5
82.0
Evaluate
*Team scores are calculated dynamically by averaging all evaluation results upon request

HMS5


HMS
Home
Challenges
Event
Reward
Team & Standings
Results
Hackathon Events
Global AI Challenge
Active
CyberSecurity Hack
Draft
Game Jam 2024
evaluation
Create New Event
Global AI Challenge
Position
Reward Name
Type
Description
1
$2,000 Cash
PRIZE
Awarded to highest scoring team.
2
$1,000 Cash
PRIZE
Awarded to the second highest scoring team
3
$500 Cash
PRIZE
Awarded to the third highest scoring team
Finalist
Excellence Cerf.
CERTIFICATE
Official recognition of tech achievement
Participant
Participant Cerf.
CERTIFICATE
Given to all teams with valid project

HMS6


HMS
Home
Challenges
Event
Reward
Team & Standings
Evaluation
Role : Judge
Hackathon Events
Global AI Challenge
Active
CyberSecurity Hack
Draft
Game Jam 2024
evaluation
Create New Event
Global AI Challenge
Evaluation forTeam: AI Wizards
Status:Pending
Numeric Score
120
Error: Score must be in [0-100]
Feedback
Project Name: "AI Powered Customer Support"
Project Link: "github.com/aiwizards/hackathon-pro"
Tools Used: Python 3.10, TensorFlow 2.15, OpenAI API
Save as draft
Confirm
8All actions are validated acccording to bussiness rules

DESING DECISION-dynamic analysis

State Diagram**:** The lifecycle of a HackathonEvent (Draft, Published, Active, Evaluation, Completed, Cancelled) is managed through a status attribute of type EventStatus (Enumeration). To support the transitions defined in the state diagram, methods such as publish(), startEvent(), endDuration(), cancel(), and finalizeResult() were added to the HackathonEvent class.

Activity Diagram**:** The Evaluate Submission activity diagram revealed the need for multi-step processing and validation. Consequently, the Evaluation class was updated with submitEvaluation(), saveAsDraft(), and validateScore() methods. To track the progress of a judge's task, an EvaluationStatus enumeration (Pending, Completed, Draft) was also introduced.

Constraints**:** A logic check identified in the activity diagram validates score is implemented as a formal constraint: {score must be in [0-100]}. This is enforced within the validateScore() method to ensure data integrity during the evaluation phase.

Class Extent & Persistence**:** To manage the set of all instances for business classes (e.g., Team, User), we implemented the class extent pattern using private static List<Class> allInstances = new ArrayList<>(). New objects are automatically added to this list within the class constructor, ensuring memory level persistence throughout the application lifecycle.

Association Classes**:** UML association classes such as Mentorship and ToolUsage are implemented as independent Java classes. For instance, the Mentorship class holds explicit references to both Mentor and HackathonEvent, along with its unique attribute availableHours.

Enumerations: UML enumerations (e.g. EventStatus, EvaluationStatus, RewardType) are implemented using Java enum keyword. This ensures that attributes like status can only hold predefined values (e.g. DRAFT, PUBLISHED and so on). public enum EventStatus { DRAFT, PUBLISHED, ACTIVE, ... }

Qualified Associations**:** The qualifier title:String used to access challenges within a HackathonEvent is implemented using a HashMap<String, Challenge>. This allows for efficient retrieval of a specific challenge object using its unique title as a key.

Mandatory & Optional Attributes: Essential fields such as name, surname ,email and password are enforced during object instantiation via constructors .This ensures that an object cannot exist in an invalid state. Fields like feedback in the evaluation class are handled via setter methods because it is implemented as optional attribute. This allows these fields to remain null or empty I f the user chooses not to provide them

Derived Attributes**:** Attributes marked with a slash (e.g., /averageScore in Submission and /memberCount in Team) are implemented as derived attributes. These values are not stored as fixed fields but are calculated dynamically via getter methods (e.g., getAverageScore()) to ensure real time accuracy and avoid data redundancy.

Abstract Classes & Unique Constraints**:** The User class is defined as abstract to support specialized roles (Organizer, Participant, etc.). To satisfy the requirement for unique emails, a class level method findByEmail() was implemented to validate user registration.

Layout**:** While standard UML hierarchs often suggests vertical inheritance, a horizontal layout was intentionally selected for the user hierarchy. This decision was made to maximize the readability of the extensive attribute and method lists and to prevent unnecessary line crossings within the diagram.

Business Constraints**:** Constraints such as the XOR Rule (preventing a user from being both a Judge and a Participant in the same event) and Team Capacity {memberCount <= maxTeamSize} are enforced through business logic within the registration and addMember() methods, respectively.